

Quick AI Read

Quiz

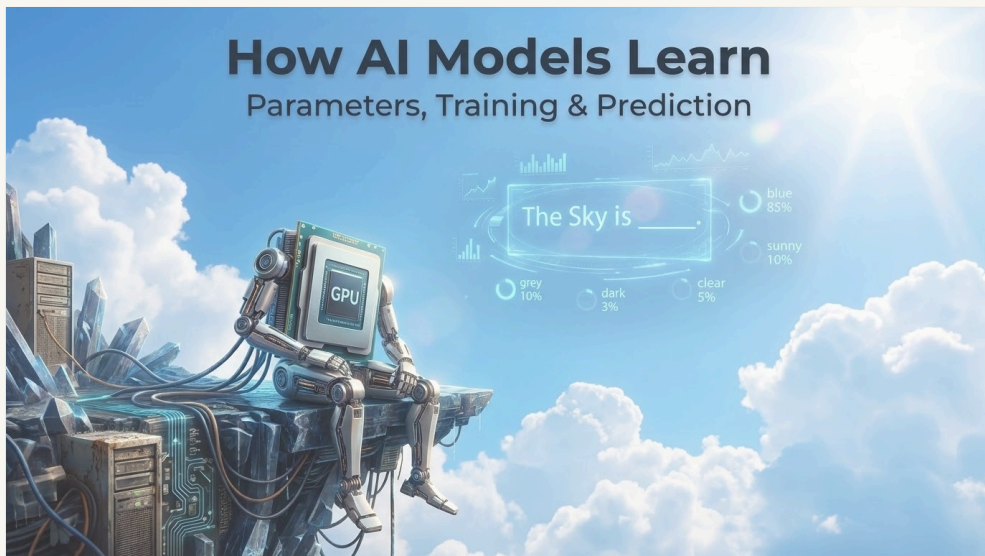
Q&A

How AI Models Learn: Parameters, Training & Prediction



By Ashutosh Anand Tiwari

Published Mar 9, 2026 · 22 min read



Here we will talk about **training**. Till now, we have discussed what happens **after training**, which is called **inference**. Now we will discuss the training of the model—how that happens and what existed before Transformer architecture came into the picture.

But Transformer architecture works with a trained neural network, so let's discuss what happens before that.

How do LLMs learn? How does training happen? What are the steps to train a neural network? What is the difference between a trained and an untrained neural network?

In easy words, an **untrained** neural network is like a newborn baby's brain, while a **trained** neural network is like an adult who has practiced a skill for years.

But does that mean when we say a **trained neural network**, what happens if we have a poorly trained neural network or an untrained network? It might give me random generations.

But what does **learning** mean in a machine?

Learning

A neural network contains a huge collection of adjustable numbers called **parameters**. During training, we repeatedly modify those parameters so that the model becomes better at a particular objective. For a neural network, learning means adjusting parameters so that future predictions become better.

Learning in a neural network is **how an AI trains itself to make accurate predictions**. The network starts by guessing answers to a problem, but it initially makes many mistakes. Every time it guesses wrong, a mathematical formula calculates how far off the guess was from the correct answer. The network then works backward to adjust the internal weights or connections between its artificial neurons.

Parameters

We know that, by mathematics, the next word is guessed or predicted. So, the mathematics happens because of the **parameters** as well. What are those parameters? **Parameters are a huge collection of numbers**.

Parameters in a neural network are the internal parts that the AI learns and adjusts during training to make accurate predictions. Think of them as millions of tiny knobs and dials inside the AI's brain. When the network is first created, these dials are set randomly, causing the AI to make poor guesses. Every time the AI practices on data, it checks its mistakes and slightly turns these knobs to improve. By the

end of training, these parameters hold the exact settings and "knowledge" the AI needs to recognize patterns, translate languages, or generate text.

or

Parameters in a neural network are the internal variables that the AI model learns and adjusts during its training process. Think of them as millions of tiny, adjustable knobs and dials inside the AI's brain. When the network makes a mistake, it turns these knobs slightly to improve.

Think of yourself as a DJ. To make better sound, you regulate knobs and adjust the sound to make the perfect setting for that program. The sound box is the **model**, and the knobs and controls that we adjust before getting the real output are like **parameters**. You can say that everything happens based on these parameters.

But in an AI neural network, who tunes these? The **model itself during training**.

The GPT-3 model has **175,000,000,000 (175B parameters)**. This is the number of parameters. GPT-4/5 has trillions of parameters. Parameters are the learned numerical values that determine the model's behavior. Patterns and knowledge learned during training are encoded across these parameters.

Remember, the embedding of a token—the array of numbers—can also contain **parameters**.

In a **neural network**, parameters are **the adjustable numbers—specifically weights and biases—that the AI fine-tunes while learning from data**. **Weights** act like volume dials that decide how important a specific input is, while **biases** act like shifting offsets that help the model make accurate decisions even when the inputs are low. Together, they function like interconnected knobs that change shape during training until the AI gets the right answers.

Do parameters store the knowledge?

The answer is **no, not exactly**. They store patterns and relationships within themselves, but not knowledge or memory in the way humans store facts. The algorithms use these learned parameters and patterns to process input and give us meaningful output.

Training Data

Training data is the collection of examples used to teach an artificial intelligence model. Think of it as the textbooks, flashcards, and practice problems you use when learning a new subject.

Remember, **parameters are inside the neural network**, and **training data is kind of the feed for the neural network**. Training data is all the possible valid information fed to the neural network.

Now let's discuss how training takes place.

Forward Pass

So suppose you put an input:

`'The Sky is ____'`

And the output comes with a higher probability for **banana [0.85]**, just for example. But you know the sky can be **blue, beautiful, or dark**, but not banana.

So it means the model itself, or the neural network, is not trained properly. That's why it is giving the wrong prediction.

So the parameters need to be adjusted so that the correct output will be predicted, and **blue** will get a higher value than **banana**.

And this is done using a **loss function**.

Loss Function

A **loss function** is a numerical way to measure how bad the prediction is. A loss function converts the quality of a prediction into a number.

Correct prediction ⇒ Small loss

Very wrong prediction ⇒ Large loss

According to the loss, the knobs—or we can say **parameters**—are adjusted. So, low loss means the model is doing well. It's a repeated loop: **predict and adjust**.

Know the keywords and concepts. This is all done using code, and the code is written behind the scenes, but the most important thing for us is to understand the concept.

A loss function is a tool that measures how wrong an AI model's predictions are compared to the true answers. Think of it as a strict but helpful teacher grading a test: the more mistakes the AI makes, the higher the **loss** score it receives.

The ultimate goal of the AI is to get this score as close to zero as possible. So, when **blue** is the answer, the loss will be low. If the prediction is **banana**, but the correct answer was **blue**, it means the loss is high. We then target those parameters that are responsible for the wrong prediction.

Coming to the **forward pass**, which is the upper layer of the loss function kind of concept.

A **forward pass** is the journey data takes through an AI model to produce a prediction. Think of it as a one-way assembly line where the input enters at the beginning, travels through various processing stations (layers), and emerges as a finished product at the end. **How It Works: "The sky is ____"** **Input:** You give the AI the phrase "The sky is". **Processing:** The AI converts these words into numbers. It passes

these numbers through its network, where it multiplies them by its learned knowledge (weights) to analyze the context. **Output:** The network calculates the probabilities for the next possible word and outputs its best guess: "blue".

Back Propagation (Backward Pass)

So, when we get the parameters causing the wrong prediction, we check and go back layer by layer and make adjustments.

Backpropagation calculates how changing each parameter would affect the loss.

In deep learning, a **backward pass** (or backpropagation) is how an AI learns from its mistakes. Imagine the AI is playing a guessing game.

Forward Pass: The AI takes the prompt "The sky is ____" and guesses "Banana". **Calculate Loss:** You tell the AI, "Wrong. The correct answer is Blue. Your guess was very far off." This error is called the **loss**. **Backward Pass:** The AI travels **backward** through its internal network. It adjusts its internal settings (called weights) so that the next time it sees "The sky is...", it is much more likely to guess "Blue" instead of "Banana".

What is Backpropagation?

Backpropagation is an algorithm used in machine learning to optimize neural networks. It works backwards from the final error, layer by layer, to error, calculate how much each weight and bias contributed to that error.

It calculates the sensitivity of the loss to each parameter (weights and biases). Mathematically, these sensitivities are called partial derivatives (gradients). Backpropagation efficiently calculates these gradients.

- At the output layer (after loss calculation):
 - How much did the parameters of Layer C contribute to the total error?
 - How much should each weight in Layer C be adjusted?
- Working backward (e.g., from Layer C to B):
 - Knowing Layer C's gradient, how much did the parameters of Layer B contribute?
 - How much should each weight in Layer B be adjusted?
- Working backward (e.g., from Layer B to A):
 - How much did the parameters of Layer A contribute?
 - ...and similarly for all preceding layers.

The sky is ...

A

B

C

banana

X WRONG PREDICTION

Correction with Loss and Backprop:

w_0

w_1

w_2

→ blue

Gradient (Sensitivity)

Gradients are simply a measure of how much a change in a specific AI parameter (like a knob or dial) will increase or decrease the AI's mistakes (the loss). Think of the AI as playing a game where it tries to guess the next word in a sentence, and every wrong guess costs points. By looking at the gradients, the AI learns exactly which "knobs" to turn, and in which direction, to make fewer mistakes next time.

The question comes: **Does backpropagation fix the weights or parameters?**

And the answer is **no**. Backpropagation calculates the **gradients**, then an optimization algorithm uses those gradients to update the parameters.

So, there are multiple algorithms that exist. **Gradient Descent** is one of them: [Link](#)



Gradient Descent Algo

-A



It is an iterative optimization algorithm used to minimize a function (like error or loss) by adjusting in the opposite direction of the gradient.

The **Gradient Descent algorithm** is how an AI learns from its mistakes to make better guesses. Imagine you are walking down a foggy mountain in the dark, trying to find the very bottom valley. Because you cannot see the path, you feel the slope of the ground with your feet and take a small step in the direction that goes downward.

In AI, the "mountain" is the amount of mistakes (called error or loss) the computer makes. For a next-word prediction task, if the AI guesses "**the sky is banana**," the error is very high because it makes no sense. The algorithm calculates exactly how wrong that guess was and adjusts the AI's internal settings (weights) step by step to reduce the error.

Eventually, after many small adjustments, the AI successfully minimizes its mistakes and correctly guesses that **the sky is blue**.

The core components of **Gradient Descent** are: **Loss Function, Gradient, and Learning Rate**. The **gradient** tells us the slope direction, and after many repetitions, the model can guess the right answer.

There are different types of optimization algorithms as well.

1. Batch Gradient Descent

This method looks at your **entire dataset** of text examples all at once before making any changes. **The Example:** The model reads thousands of sentences like "The sky is blue," "The sky is gray," and "The sky is cloudy." It calculates the total error across all these sentences combined. Only after analyzing every single sentence does it update its internal settings once.

Pros & Cons: It takes a very steady, accurate path to the right answer, but it is extremely slow and uses a lot of computer memory.

2. Stochastic Gradient Descent (SGD)

This method updates the model after looking at **just one random example** at a time. **The Example:** The model looks at a single sentence: "The sky is blue." It immediately changes its settings based on that one sentence. Then it looks at "The sky is dark" and changes its settings again right away.

Pros & Cons: It is incredibly fast and uses very little memory. However, because it jumps from sentence to sentence, its path to the correct answer is chaotic, erratic, and bounces around a lot.

3. Mini-Batch Gradient Descent

This method is a hybrid that splits the dataset into **small groups (batches)** of examples. **The Example:** The model splits its dataset into small groups of 32 sentences. It reads the first 32 sentences about the sky, calculates the average error for that specific group, and updates its settings. Then it moves on to the next group of 32 sentences.

Pros & Cons: This is the industry standard. It combines the speed of SGD with the steady accuracy of Batch Gradient Descent, making it the most efficient way to train modern AI models.

So if we sum up, we get **sample data**, then the **forward pass** happens and it predicts something. Then we calculate the **loss**, then **backpropagation** happens, and then **optimization** comes into the picture, including **Gradient Descent**. Then the model becomes better.

This is what happens during training, and we keep doing this again and again for a huge amount of training data.

So when any company says, **we have this model having this many parameters, trained on billions or trillions of data**, now you are getting it, right? How does that happen? It's all about patterns. Multiple text inputs are given, the model guesses the output, and it starts recognizing the sequence of words. So it learns automatically, and after some iterations, it can generate better answers than humans in some tasks.

And this process takes a lot of computational power and GPUs because it is a repeated process.

And whatever we learned above is called **self-supervised learning**.

Self-Supervised Learning



Self-supervised learning is a type of machine learning where a computer teaches itself without needing humans to label the data. Instead of a person marking images as "cat" or "dog," the system looks at raw, unlabeled data—like millions of internet sentences or photos—and hides parts of it from itself. It then tries to guess the missing pieces, such as predicting the next word in a sentence or filling in a blurry part of a picture. By practicing this guessing game billions of times, the AI naturally figures out the underlying patterns, structures, and meanings of the data all on its own.

Okay, now let's learn about some terminologies. When we hear these, we will not get confused.

Dataset: All the training data.

Batch: A group of examples processed together. Because GPUs can do parallel processing, we process data in batches. If we do it sequentially, it will take so much time.

Training Step: One optimization update. You remember the forward and backward pass—refer above.

Epoch: One complete pass through the training dataset, and it is different from a batch.

An **epoch** in AI is one complete round of training where the machine learning model gets to look at the entire training dataset exactly once. Think of it like reading a textbook to prepare for an exam; reading the whole book from cover to cover represents one epoch. Because AI models cannot learn everything from just one look, they usually need to go through the same dataset for dozens or hundreds of epochs. During each epoch, the model analyzes the data, makes mistakes, and adjusts its internal settings to become smarter and more accurate for the next round.

Difference Between Training and Inference

Training is the process of teaching an AI model how to learn, while **inference** is the process of using that trained model to make real-world predictions. During training, you feed the AI huge amounts of data and correct its mistakes (**remember forward and backward pass**) so it can figure out the patterns and rules.

Once the AI finishes learning and graduates, it enters the **inference phase**. In this step, you give the model completely new data, and it instantly applies its learned knowledge to give you an answer, like a chatbot replying to your question or a face scanner unlocking your phone.

Remember, **training consumes more power and has a higher financial cost**.

Generalization

The ability to perform well on examples it did not directly memorize is called **generalization**.

Generalization in AI is the ability of a machine learning model to apply what it has learned from past data to brand-new, unseen situations. It means the AI does not just memorize the exact examples it was trained on; instead, it understands the underlying concepts and rules. For example, if an AI is trained on images of sunny days, it learns that the sky is blue. If it later sees a picture of a stormy day and correctly predicts that the **sky is gray**, it has successfully generalized. It applied the broad concept of "sky" to a completely new context rather than just repeating its original training. Overfitting is just opposite of it, a useful model should generalize.

Good learning means discovering patterns that remain useful beyond the exact training examples.

Overfitting

A model is **overfitting** when it becomes extremely good at training examples but performs poorly on new examples. In other words,

Overfitting in AI happens when a computer learns the training data too perfectly, memorizing specific examples instead of understanding the big picture. Because it memorizes instead of learning, it can fail when facing new information.

Imagine you teach an AI the phrase **"the sky is blue"** using 100 pictures of sunny days. If the AI overfits, it decides that the sky can *only* be blue. When you show it a picture of a stormy night and ask, **"the sky is ____"**, the overfitted AI will still confidently answer **"blue"** because it memorized a rigid rule instead of learning that the weather changes.

Distributed Training

Training a frontier-scale model is a large distributed system problem as much as it is a machine learning problem, in other words.

Distributed training in AI means splitting the massive job of teaching an AI model across multiple computers (called nodes) instead of using just one. Think of it like baking thousands of cookies for a big party. If one person does it alone, it takes days. If you invite ten friends and give everyone a mixing bowl, you finish in a few hours.

In AI, a single computer might take weeks to learn from billions of data pieces and could grow too hot or run out of memory. By breaking the huge dataset or the AI model itself into smaller pieces, the computers can work at the same time, talk to each other to share what they learned, and finish the training in hours instead of months.

You can also buy/rent an **H100 GPU**, and its price can be around **₹25 lakh to ₹40 lakh**. Search online.

Okay, now let's recap the things and try to understand. Remember, all embedding values are parameters, and those embeddings adjust in backward and forward loss functions in learning, and values go up and down.

Close your eyes and from starting to end, imagine the picture of how things are written, and whatever you don't understand, go and read. It's all magic and its beauty because it is beyond the human brain. How things are written, everything you can learn and read, explore, research, and share here.

Now we also know models learn. The model understands, as humans understand and learn. Does a machine do the same? Till now, we have seen machines predict! Think Think Think.....

Stopping here and leaving so many searchable questions. Be in front of the mirror and answer these, or explain them to your mate. What are parameters?

- **H**ow do embeddings learn?
- **H**ow do parameters change?
- **W**hat happens during a forward pass?
- **W**hat exactly does backpropagation do?
- **H**ow does gradient descent work?
- **H**ow does an LLM learn from data?
- **H**ow does a model generalize?
- **W**hy does overfitting happen?
- **D**oes a machine really understand, or just predict?

Continue Exploring

COLLECTION

Chapter 1: Into Programming

Get all digital notes here about namaste node js

COLLECTION

HLD High Level Design Overview

Continue exploring notes from the digital garden.



Curated by

Ashutosh Anand Tiwari

Planting free, open-source tech notes and engineering guides so developers worldwide can learn, revise, and excel without paywalls.

Follow on Instagram

Star on GitHub ★

QUICK NAVIGATION

Course Notes

Products

Join Namaste Dev Club

Tech Blogs

Support Garden

CONNECT & SOCIALS

